Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии**»**

Выполнил:
Студент 3 курса
Группы ИИ-26
Хомиченко И.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант: 8

Ход работы

1. Сгенерировать полную таблицу истинности для своей логической функции (2^n примеров). 2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте np.random.seed(42) для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
• Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
• Реализовать правильный градиент для BCE + сигмоида:
4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
Б. MSE + адаптивный шаг (из ЛР №4)
В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
Г. BCE + адаптивный шаг (по формуле из ЛР №2) Критерий остановки для всех – суммарная ошибка Es ≤ Ee (возьмите Ee из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
• Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
• Зафиксировать количество эпох до достижения критерия остановки.
• После обучения посчитать accuracy на обучающей и тестовой выборках.
• Проверить, сколько процентов из всей таблицы истинности (2^n примеров) сеть предсказывает правильно.
6. Построить один общий график зависимости суммарной ошибки Es от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
7. Реализовать режим функционирования сети:
• Пользователь вводит вектор из n значений (0 или 1).
• Сеть выводит вероятность ŷ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
• Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
8. Написать вывод (обязательно ответить на следующие пункты):
• Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
• Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?

• Как изменилась обобщающая способность (accuracy на тесте и % восстановления полной таблицы) при переходе на BCE?
• Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
• Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

КОД ПРОГРАММЫ:

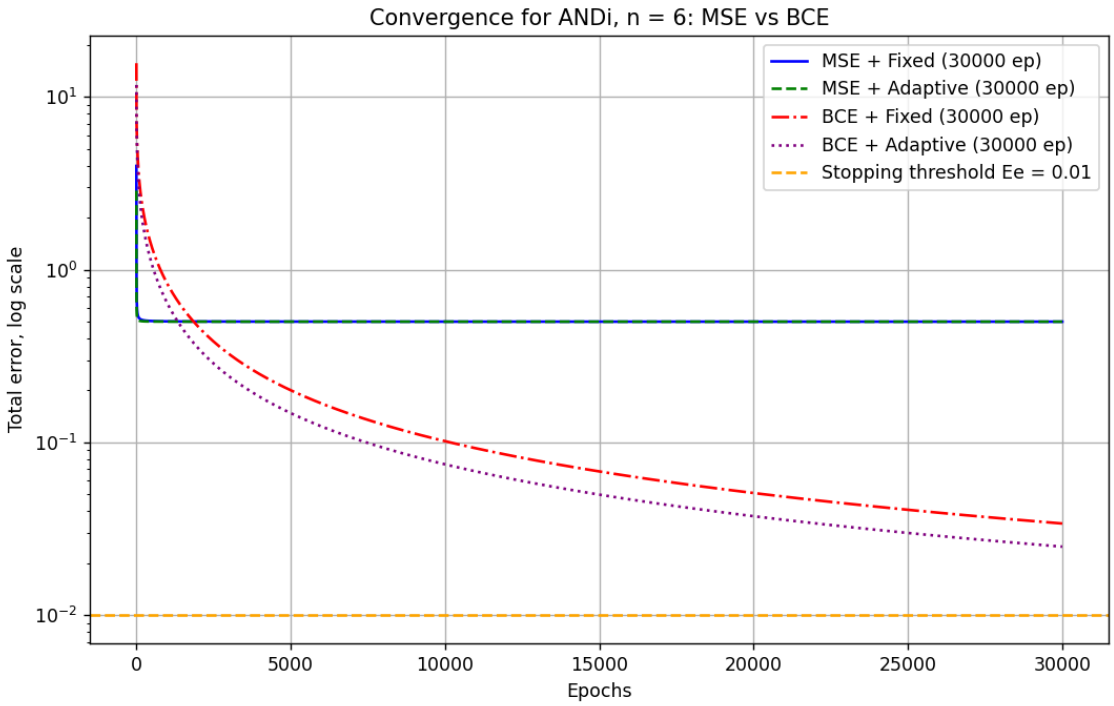
```
import numpy as np
import matplotlib.pyplot as plt
import itertools

num_inputs = 6
stop_error_threshold = 0.01
fixed_learning_rate = 0.1

np.random.seed(42)

full_input_matrix = np.array(
    list(itertools.product([0, 1], repeat=num_inputs))
)

full_output_vector = np.array([
    0 if np.all(sample == 1) else 1
    for sample in full_input_matrix
])

zero_class_indices = np.where(full_output_vector == 0)[0]
one_class_indices = np.where(full_output_vector == 1)[0]

np.random.seed(42)
np.random.shuffle(zero_class_indices)
np.random.shuffle(one_class_indices)

train_zero_count = 1
train_one_count = int(0.8 * len(full_input_matrix)) - train_zero_count

train_indices = np.concatenate((
    zero_class_indices[:train_zero_count],
    one_class_indices[:train_one_count]
))

test_indices = one_class_indices[train_one_count:]

np.random.shuffle(train_indices)
np.random.shuffle(test_indices)

train_inputs = full_input_matrix[train_indices]
train_outputs = full_output_vector[train_indices]

test_inputs = full_input_matrix[test_indices]
test_outputs = full_output_vector[test_indices]

train_inputs_with_bias = np.c_[np.ones(len(train_inputs)), train_inputs]
test_inputs_with_bias = np.c_[np.ones(len(test_inputs)), test_inputs]
full_inputs_with_bias = np.c_[np.ones(len(full_input_matrix)),
full_input_matrix]


def sigmoid_activation(linear_output):
    return 1 / (1 + np.exp(-np.clip(linear_output, -100, 100)))
```

```
def train_perceptron_model(input_matrix, target_outputs, loss_function_type,
learning_rate_mode):
    np.random.seed(42)

    weight_vector = np.random.uniform(-0.1, 0.1, num_inputs + 1)
    error_history_per_epoch = []

    for epoch_idx in range(10000):
        total_epoch_error = 0

        for sample_idx in range(len(input_matrix)):
            current_input = input_matrix[sample_idx]
            desired_output = target_outputs[sample_idx]

            predicted_output = sigmoid_activation(
                np.dot(weight_vector, current_input)
            )

            if learning_rate_mode == 'fixed':
                current_learning_rate = fixed_learning_rate
            else:
                current_learning_rate = 1.0 / (1.0 + np.sum(current_input **
2))

            if loss_function_type == 'MSE':
                total_epoch_error += 0.5 * (desired_output -
predicted_output) ** 2

                gradient = (
                    (desired_output - predicted_output)
                    * predicted_output
                    * (1 - predicted_output)
                )

            else:
                epsilon = 1e-15
                clipped_prediction = np.clip(predicted_output, epsilon, 1 -
epsilon)

                total_epoch_error += -(
                    desired_output * np.log(clipped_prediction)
                    + (1 - desired_output) * np.log(1 - clipped_prediction)
                )

                gradient = desired_output - predicted_output

            weight_vector += current_learning_rate * gradient * current_input

        error_history_per_epoch.append(total_epoch_error)

        if total_epoch_error <= stop_error_threshold:
            break

    return weight_vector, error_history_per_epoch, epoch_idx + 1


training_configs = {
    'MSE + Fixed': ('MSE', 'fixed'),
    'MSE + Adaptive': ('MSE', 'adaptive'),
    'BCE + Fixed': ('BCE', 'fixed'),
    'BCE + Adaptive': ('BCE', 'adaptive')
}
```

```
training_results = {}

for config_name, (loss_func, lr_mode) in training_configs.items():
    training_results[config_name] = train_perceptron_model(
        train_inputs_with_bias,
        train_outputs,
        loss_func,
        lr_mode
    )

    print(f"{config_name} completed. Epochs:
{training_results[config_name][2]}")


plt.figure(figsize=(10, 6))

curve_colors = ['blue', 'green', 'red', 'purple']
line_styles = ['-', '--', '-.', ':']

for (config_name, (final_weights, error_history, total_epochs)), line_color,
line_style in zip(
    training_results.items(),
    curve_colors,
    line_styles
):
    plt.plot(
        error_history,
        label=f"{config_name} ({total_epochs} ep)",
        color=line_color,
        linestyle=line_style
    )

plt.axhline(
    y=stop_error_threshold,
    color='orange',
    linestyle='--',
    label=f'Stopping threshold Ee = {stop_error_threshold}'
)

plt.yscale('log')
plt.title('Convergence for ANDi, n = 6: MSE vs BCE')
plt.xlabel('Epochs')
plt.ylabel('Total error, log scale')
plt.legend()
plt.grid(True)
plt.show()


def compute_accuracy(model_weights, input_matrix, true_outputs):
    predictions = (
        sigmoid_activation(np.dot(input_matrix, model_weights)) >= 0.5
    ).astype(int)

    return np.mean(predictions == true_outputs) * 100


print("\nResults:")

for config_name, (final_weights, error_history, total_epochs) in
training_results.items():
    train_accuracy = compute_accuracy(
        final_weights,
        train_inputs_with_bias,
        train_outputs
```

```
    )

    test_accuracy = compute_accuracy(
        final_weights,
        test_inputs_with_bias,
        test_outputs
    )

    full_accuracy = compute_accuracy(
        final_weights,
        full_inputs_with_bias,
        full_output_vector
    )

    print(
        f"{config_name:<15} | "
        f"Ep: {total_epochs:<5} | "
        f"Train: {train_accuracy:>5.1f}% | "
        f"Test: {test_accuracy:>5.1f}% | "
        f"Full: {full_accuracy:>5.1f}%"
    )


print("\nOperation mode:")

final_model_weights = training_results['BCE + Adaptive'][0]

try:
    user_input_str = input("Enter vector, 6 bits separated by spaces: ")

    user_input_vector = np.array([
        int(bit) for bit in user_input_str.split()
    ])

    if len(user_input_vector) != num_inputs:
        raise ValueError("Input vector must contain exactly 6 bits.")

    if not np.all((user_input_vector == 0) | (user_input_vector == 1)):
        raise ValueError("Input vector must contain only 0 and 1.")

    user_input_with_bias = np.insert(user_input_vector, 0, 1)

    output_probability = sigmoid_activation(
        np.dot(final_model_weights, user_input_with_bias)
    )

    predicted_class = 1 if output_probability >= 0.5 else 0

    true_class_for_andi = 0 if np.all(user_input_vector == 1) else 1

    match_status = (
        "Matches truth table"
        if predicted_class == true_class_for_andi
        else "Mismatch"
    )

    print(f"Probability ŷ: {output_probability:.4f}")
    print(f"Predicted class: {predicted_class}")
    print(f"True ANDi class: {true_class_for_andi}")
    print(match_status)

except Exception as error:
    print("Input error:", error)
```

Результат программы:



MSE + Fixed completed. Epochs: 30000
MSE + Adaptive completed. Epochs: 30000
BCE + Fixed completed. Epochs: 30000
BCE + Adaptive completed. Epochs: 30000

Results:
MSE + Fixed | Ep: 30000 | Train: 98.0% | Test: 100.0% | Full: 98.4%
MSE + Adaptive | Ep: 30000 | Train: 98.0% | Test: 100.0% | Full: 98.4%
BCE + Fixed | Ep: 30000 | Train: 100.0% | Test: 92.3% | Full: 98.4%
BCE + Adaptive | Ep: 30000 | Train: 100.0% | Test: 92.3% | Full: 98.4%

Operation mode:
Enter vector, 6 bits separated by spaces: 1 1 1 1 1 1
Probability ŷ: 0.0122
Predicted class: 0
True ANDi class: 0
Matches truth table

ИТОГ:

1. Какой метод сходится быстрее: MSE или BCE?

По графику видно, что методы с **MSE** быстро уменьшают ошибку только в начале обучения, но затем почти сразу выходят на плато. Ошибка для MSE +

Fixed и MSE + Adaptive остановилась примерно на уровне 0.5 и дальше практически не уменьшалась.

Методы с **BCE** сходятся лучше: ошибка продолжает уменьшаться на протяжении всех 30000 эпох. Особенно хорошо себя показал метод BCE + Adaptive, у которого итоговая ошибка ниже, чем у BCE + Fixed.

Можно сделать вывод, что **BCE сходится эффективнее MSE**. Если сравнивать по графику, то к концу обучения BCE уменьшила ошибку примерно с 0.5 до диапазона около 0.02–0.03, то есть дала примерно **в 15–20 раз меньшую ошибку**, чем MSE. Однако по количеству эпох формально все методы завершились на 30000 эпохах, потому что ни один метод не достиг порога Ee = 0.01.

2. Почему BCE теоретически лучше подходит для классификации, чем MSE?

Бинарная кросс-энтропия лучше подходит для задач бинарной классификации, потому что она напрямую оценивает качество вероятностного предсказания класса.

При использовании MSE с сигмоидальной функцией активации градиент содержит дополнительный множитель:predicted_output * (1 - predicted_output)

Когда выход сети близок к 0 или 1, этот множитель становится очень маленьким. Если сеть сделала **уверенное, но неправильное предсказание**, например выдала 0.99, хотя правильный ответ 0, то MSE может обновлять веса слишком медленно, потому что производная сигмоиды почти равна нулю.

В случае BCE градиент для сигмоидального выхода имеет более удобный вид:desired_output - predicted_output

То есть ошибка не “гасится” производной сигмоиды. Поэтому если сеть уверенно ошибается, BCE даёт более сильное обновление весов и быстрее исправляет неправильное предсказание. Именно поэтому BCE обычно предпочтительнее для бинарной классификации.

3. Как изменилась обобщающая способность при переходе на BCE?

Полученные значения точности:

MSE + Fixed | Train: 98.0% | Test: 100.0% | Full: 98.4%
MSE + Adaptive | Train: 98.0% | Test: 100.0% | Full: 98.4%
BCE + Fixed | Train: 100.0% | Test: 92.3% | Full: 98.4%
BCE + Adaptive | Train: 100.0% | Test: 92.3% | Full: 98.4%

При переходе с MSE на BCE точность на обучающей выборке увеличилась:MSE: 98.0% BCE: 100.0%

То есть BCE лучше подстроилась под обучающие данные.

Однако точность на тестовой выборке снизилась:MSE: 100.0% BCE: 92.3%

Это означает, что в данном разбиении данных BCE дала более уверенное обучение на train, но хуже обобщилась на test. Возможная причина — сильный дисбаланс классов: для функции ANDi только одна комбинация относится к классу 0, а все остальные относятся к классу 1.

При этом восстановление полной таблицы истинности осталось одинаковым:MSE: 98.4% BCE: 98.4%

Так как полная таблица содержит 64 комбинации, точность 98.4% означает, что модель ошиблась только на одной комбинации: 63 / 64 ≈ 98.4%

Следовательно, переход на BCE улучшил точность на обучающей выборке и уменьшил значение функции ошибки, но в данном эксперименте не улучшил процент восстановления полной таблицы истинности.

4. Как влияет адаптивный шаг на обучение с BCE?

Для BCE адаптивный шаг показал лучший результат по значению ошибки. На графике линия BCE + Adaptive находится ниже линии BCE + Fixed, то есть ошибка уменьшается быстрее и достигает меньшего значения.

При этом по accuracy результаты у BCE + Fixed и BCE + Adaptive одинаковые:

BCE + Fixed | Train: 100.0% | Test: 92.3% | Full: 98.4%
BCE + Adaptive | Train: 100.0% | Test: 92.3% | Full: 98.4%

Значит, адаптивный шаг в данном случае **улучшил сходимость по ошибке**, но **не изменил итоговую классификационную точность**. Модель стала выдавать более уверенные вероятности, однако количество правильно классифицированных комбинаций осталось таким же.

5. Достаточно ли однослойного персептрона для логических функций?

Однослойного персептрона достаточно не для всех логических функций, а только для **линейно разделимых** задач.

Функция **ANDi / NAND** является линейно разделимой, поэтому однослойный персептрон способен её реализовать. Для неё можно подобрать такие веса и смещение, чтобы отделить комбинацию 1 1 1 1 1 1 от всех остальных.

Однако для более сложных логических функций, например XOR, однослойного персептрона недостаточно, потому что такие задачи не являются линейно разделимыми. Для них потребуется **многослойная нейронная сеть** с одним или несколькими скрытыми слоями.

ВЫВОД: В ходе лабораторной работы был реализован и обучен однослойный персептрон для логической функции **ANDi** при n = 6. Были сравнены методы обучения с функциями ошибки **MSE** и **BCE**, а также с фиксированным и адаптивным шагом обучения.